

Task 2

Reconnaissance, Information Gathering, Network Scanning & Enumeration

Student Name

Akshita Karn

Programme

B.Sc. Cyber Forensics

University

Galgotias University

A technical report documenting reconnaissance, information gathering, network scanning and enumeration activities carried out in an authorized virtual laboratory environment as part of the Cyber Security & Ethical Hacking Internship programme.

1. Introduction & Objectives

1.1 Background

Task 1 of this internship established the foundational cybersecurity laboratory used throughout the programme. This included setting up a Kali Linux virtual machine, developing familiarity with core Linux fundamentals, reviewing basic networking concepts, and building a basic Python system-information utility. That environment forms the technical foundation for the work carried out in Task 2.

Task 2 focuses on the first phase of a structured security assessment: **reconnaissance and enumeration**. This phase involves gathering information about a target system or network prior to any deeper technical testing, and is a standard starting point in ethical hacking and penetration-testing methodologies.

1.2 Scope of Work

The activities performed in this task cover both passive and active information-gathering techniques, applied against an authorized lab target within a controlled virtual network. The following areas were explored:

- Passive reconnaissance – gathering publicly available information without directly interacting with the target.
- Active reconnaissance – directly probing the target system to observe its responses.
- Asset discovery – identifying whether a host is live and reachable on the network.
- Information gathering – collecting domain, DNS and registration-related information.
- Network scanning – examining a target for open, closed or filtered ports.
- Service and version identification – attempting to determine what services are running.
- OS detection – attempting to infer the target operating system from network responses.
- DNS enumeration – resolving domain names and reviewing associated DNS records.
- Network traffic analysis – capturing and reviewing live packets using Wireshark.
- Automation using Python – scripting basic scanning tasks and generating structured reports.

1.3 Objectives

- 1 Discover active network hosts.
- 2 Perform basic port scanning.
- 3 Identify available services and versions.
- 4 Attempt operating-system detection.
- 5 Perform DNS and domain information gathering.
- 6 Analyze network traffic using Wireshark.
- 7 Use Netcat for basic connectivity / banner-grabbing testing.
- 8 Automate network scanning using Python.
- 9 Generate structured CSV, JSON and HTML results.
- 10 Document findings professionally.

2. Lab Environment & Tools

2.1 Lab Environment

All activities were carried out in a controlled, isolated virtual laboratory. The environment consisted of a Kali Linux virtual machine running inside VirtualBox, connected to a local/private lab network. No public or third-party systems were scanned or tested.

Parameter	Value
Attacking / scanning machine	Kali Linux (VirtualBox VM)
Kali interface address used during the lab	10.0.2.15/24
Authorized target used for testing	10.0.0.2
Network type	Local / private virtual lab network

2.2 Tools Used

The following tools were used to carry out the reconnaissance and enumeration activities in this task:

Tool	Purpose	Task Performed
Kali Linux	Penetration-testing operating system	Base platform for all reconnaissance activities
Nmap	Network scanning and enumeration	Host discovery, port scanning, service/version detection, OS detection
Wireshark	Network traffic capture and analysis	Captured and reviewed live packets on the lab network
Netcat	Connectivity testing and banner grabbing	Tested TCP connectivity to the target on port 80
WHOIS	Domain registration lookup	Retrieved domain registration information
dig	DNS record lookup	Queried DNS records for a domain
nslookup	DNS resolution utility	Resolved domain names to IP addresses
tracert	Network path mapping	Traced the network path to a destination host
Python 3	Scripting and automation	Automated scanning and generated structured scan reports
Git / GitHub	Version control	Tracked and stored project source code and outputs

3. Reconnaissance & Nmap Results

This section documents the actual Nmap activities performed against the authorized lab target (10.0.0.2). All results below reflect the real output observed during testing; no results, open ports or services have been invented.

3.1 Host Discovery

```
nmap -sn 10.0.0.2
```

This scan was used to determine whether the target host was live and reachable on the lab network. The scan produced host-availability information for the specified target, confirming the target's presence on the network prior to further testing.

3.2 Port Scanning

```
nmap 10.0.0.2
```

A default Nmap scan was run against the target, scanning the 1000 most common TCP ports. The scan reported that these ports were **filtered / gave no response**, meaning Nmap could not determine whether the ports were open or closed based on the responses received.

3.3 Service & Version Detection

```
nmap -sV 10.0.0.2
```

Service and version detection was attempted using the **-sV** flag. Because the scanned ports did not return usable responses, **no services or version information could be identified** for the target.

3.4 OS Detection

```
sudo nmap -O 10.0.0.2
```

Nmap's OS detection feature was attempted against the target. Reliable operating-system identification was **not possible**, as there were no suitable open or closed ports and insufficient responses from the target to support a confident fingerprint. No operating system, device type or vendor was confirmed as a result of this scan.

Note: Screenshots documenting the above Nmap activities are included in the project evidence submitted alongside this report.

4. DNS, Traceroute, Netcat & Wireshark

4.1 DNS Enumeration

The following commands were used to gather DNS and domain information:

```
nslookup google.com
dig google.com
whois google.com
```

These tools were used to demonstrate standard DNS and domain information-gathering techniques:

- **nslookup** – resolves a domain name to its associated IP address(es).
- **dig** – queries authoritative DNS records for a domain in greater detail.
- **whois** – retrieves domain registration information, including registrar details.

4.2 Traceroute

```
traceroute google.com
```

Traceroute was used to observe the network path taken toward the destination host, showing the sequence of intermediate hops between the Kali Linux machine and the destination.

4.3 Netcat

```
nc -v 10.0.0.2 80
```

Netcat was used to test basic TCP connectivity to the lab target on port 80. The reverse host lookup for the target failed / the host was reported as unknown, and the connection attempt to port 80 was **refused**. This demonstrates the use of Netcat for basic connectivity and banner-grabbing testing; it does not indicate that any service banner was successfully retrieved from the target in this instance.

4.4 Wireshark Packet Capture

Wireshark was used to capture and review live network traffic on the lab network. The capture was configured and executed as follows:

- Capture interface: **eth0**
- Traffic generated using: **ping -c 4 google.com**
- Approximately **19 packets** were observed during the capture.
- The capture was saved as **wireshark_capture.pcapng.gz**.

This activity demonstrated the basic workflow of live traffic capture and packet-level analysis, allowing individual packets generated by the ping traffic to be inspected directly.

5. Python Automation & Project Outputs

5.1 Python Network Scanner

To automate basic network and port scanning, a single Python script, **python_scanner.py**, was developed. This script consolidates the scanning and reporting logic used in this task; separate modules (e.g. individual host-discovery, port-scanner, DNS-lookup, banner-grabber or report-generator scripts) were not implemented as part of this task.

- Python version used: **Python 3.14.7**
- Performed an initial scan of common ports on the target.
- Performed an extended scan across ports 1–100 on the target.
- No open ports were identified on the tested target based on the scan results.

5.2 Generated Report Files

The Python scanner produced results in three structured output formats:

File	Format	Purpose
scan_results.csv	CSV	Structured tabular data suitable for spreadsheet review or further analysis
scan_results.json	JSON	Machine-readable structured data suitable for integration with other tools
scan_results.html	HTML	Human-readable report suitable for direct viewing in a browser

5.3 Project Evidence

The following supporting evidence was produced and collected during this task:

- 9 Task 2 screenshots documenting the activities described in this report.
- Network topology diagram of the lab environment.
- Reconnaissance and network enumeration workflow diagram.
- Wireshark packet capture file (wireshark_capture.pcapng.gz).
- README documentation for the project.

6. Findings, Limitations, Ethics & Conclusion

6.1 Key Findings

- The target (10.0.0.2) was reachable for reconnaissance testing on the lab network.
- Nmap scans showed the 1000 scanned TCP ports as filtered, with no usable response.
- No services or version information could be identified through the **-sV** scan.
- OS detection using Nmap's **-O** flag was inconclusive.
- DNS and domain-information tools (nslookup, dig, whois) successfully demonstrated standard information-gathering techniques.
- Wireshark successfully captured live network traffic generated by a ping test.
- Netcat testing demonstrated a refused connection on TCP port 80.
- The Python automation script successfully generated CSV, JSON and HTML scan outputs.

6.2 Limitations

- All testing was performed in a controlled lab environment rather than a production or live network.
- The target did not expose sufficient open or closed ports for reliable service or OS identification.
- The results reported are limited to the specific target and network conditions used during this task.
- The Python implementation is a basic reconnaissance scanner, not a complete enterprise-grade penetration-testing framework.

6.3 Ethical & Legal Considerations

Reconnaissance and scanning activities should only ever be performed on systems for which explicit permission has been obtained. Unauthorized scanning of systems or networks may violate organizational policies, terms of service, or applicable laws. All activities described in this report were performed strictly for educational purposes within an authorized, isolated laboratory environment, and no external or third-party systems were scanned or accessed.

6.4 Conclusion

Task 2 provided practical, hands-on experience across the core stages of network reconnaissance and enumeration, including host discovery, port scanning, service enumeration, OS detection, DNS information gathering, packet analysis, Netcat-based connectivity testing, and Python-based automation and reporting. Together, these activities form a foundational skill set that supports future work in vulnerability assessment and penetration testing.

6.5 Project Deliverables

- Python source code (python_scanner.py)
- CSV report (scan_results.csv)
- JSON report (scan_results.json)
- HTML report (scan_results.html)
- Nmap scan evidence
- DNS enumeration evidence
- Wireshark packet capture (wireshark_capture.pcapng.gz)
- Network topology diagram
- Reconnaissance workflow diagram
- Screenshots (9 total)
- README documentation
- Technical Report (PDF)